

Gradient Boosting Model Home Credit Default Risk

Code Explanation & Technical Guide

home_credit.py

1. Project Overview

This project builds a supervised machine learning pipeline to predict whether a loan applicant will default. The dataset comes from the Home Credit Default Risk Kaggle competition and contains a rich set of application features plus related financial history tables.

The primary evaluation metric is ROC-AUC. The pipeline covers: exploratory data analysis, preprocessing, feature engineering from secondary tables, a baseline comparison of four model families, Optuna hyperparameter tuning, model interpretation via SHAP values, and generation of a Kaggle submission file.

High-level pipeline

Step	Function	Purpose
1	load_main_table()	Load application_train.csv
2	load_supplementary_tables()	Optionally load bureau, prev_application, etc.
3	run_eda()	Target distribution, missing values, correlations
4	preprocess()	Encode categoricals, merge secondary features
5	compare_models()	LR vs RF vs LightGBM vs XGBoost vs CatBoost
6	tune_lgbm()	Optuna hyperparameter search
7	train_lgbm_cv()	Full 5-fold CV with OOF predictions
8	Evaluation plots	ROC, PR curve, confusion matrix, SHAP
9	make_submission()	Generate Kaggle submission CSV

2. Data & Setup

Required files

Place downloaded Kaggle CSV files in a data/ subfolder relative to home_credit.py. Only application_train.csv is required to run the full pipeline. The secondary tables are optional but improve AUC via feature engineering.

File	Status	Description
application_train.csv	Required	307,511 rows, 122 features + TARGET
application_test.csv	Optional	Required only for Kaggle submission
bureau.csv	Optional	Credit bureau history per applicant
previous_application.csv	Optional	Previous loan applications
bureau_balance.csv	Optional	Monthly bureau balance history
installments_payments.csv	Optional	Installment payment history
credit_card_balance.csv	Optional	Credit card monthly balance
POS_CASH_balance.csv	Optional	POS and cash loan monthly balance

3. Exploratory Data Analysis (run_eda)

Target distribution

The dataset is highly imbalanced: approximately 8% of applicants defaulted (TARGET=1). `run_eda()` plots a bar chart of class counts and prints the default rate. This imbalance must be addressed - all models use `class_weight='balanced'`.

Missing value summary

A horizontal bar chart shows the top 30 features by missing rate. Many external data columns have 40-70% missing values. These are handled by median imputation in the preprocessing step. Features with > 50% missing are noted in the printout.

Correlation heatmap

Selects the top 20 numeric features by variance and computes pairwise Pearson correlations. A lower-triangular heatmap (masked upper triangle) is saved to outputs/. This helps spot multicollinearity and redundant features before modelling.

4. Feature Engineering from Secondary Tables

`engineer_bureau_features(bureau)`

Aggregates the `bureau.csv` table at the `SK_ID_CURR` (applicant) level. Each applicant can have multiple bureau records (one per credit line). The following aggregate features are created:

Feature	Description
<code>bureau_count</code>	Total number of credit bureau entries
<code>bureau_active_count</code>	Number of currently active credits
<code>bureau_credit_sum</code>	Total credit amount across all bureaus
<code>bureau_debt_sum</code>	Total outstanding debt
<code>bureau_overdue_sum</code>	Total overdue amount
<code>bureau_days_overdue</code>	Maximum days past due (worst account)
<code>bureau_debt_ratio</code>	Debt / (credit_sum + 1) - leverage proxy

`engineer_prev_app_features(prev)`

Aggregates `previous_application.csv` similarly. Key features include:

Feature	Description
<code>prev_app_count</code>	Total previous loan applications
<code>prev_approved_count</code>	Number of approved applications
<code>prev_refused_count</code>	Number of refused applications
<code>prev_amt_credit_mean</code>	Average credit amount applied for
<code>prev_days_decision_mean</code>	Average days since decision
<code>prev_approval_rate</code>	Approved / total - credit history quality

5. Preprocessing (preprocess)

Categorical encoding strategy

Two encoding strategies are applied depending on cardinality:

- Binary categoricals (≤ 2 unique values): Label encoding (0/1).
- Multi-class categoricals (> 2 unique): One-hot encoding with `drop_first=True` to avoid dummy trap.

All encoding is done on the training data and reapplied consistently to test data via column reindexing with `fill_value=0`.

Missing value imputation

A sklearn `SimpleImputer(strategy='median')` is applied after encoding. Median imputation is more robust to outliers than mean imputation, which matters here because financial variables (income, credit amounts) tend to have heavy right-skewed distributions.

Leakage prevention

SK_ID_CURR (applicant ID) and TARGET are dropped before modelling. A `KNOWN_LEAKAGE_COLS` list is provided for users to add any features found to be leaking future information during analysis.

6. Model Comparison (compare_models)

Four model families are evaluated using 5-fold stratified cross-validation. Stratification ensures each fold has the same class ratio as the full dataset, which is important given the 8% default rate.

Model	Settings	Notes
Logistic Regression	sklearn Pipeline (impute + scale + LR)	Interpretable baseline; $C=0.1$ for regularisation
Random Forest	100 trees, <code>max_depth=8</code>	Non-linear baseline; <code>class_weight=balanced</code>
LightGBM	300 rounds, <code>lr=0.05</code> , <code>leaves=63</code>	Fast gradient boosting; handles missing natively
XGBoost	300 rounds, <code>lr=0.05</code> , <code>max_depth=6</code>	Alternative gradient boosting framework
CatBoost	300 iters, <code>lr=0.05</code> , <code>depth=6</code>	Handles categoricals natively; if installed

7. Hyperparameter Tuning (tune_lgbm)

Optuna is a Bayesian optimisation framework that intelligently explores the hyperparameter space using Tree-structured Parzen Estimator (TPE) sampling. It is more efficient than grid search for large parameter spaces.

Parameter	Search Range	Effect
<code>learning_rate</code>	0.01 - 0.1 (log scale)	Step size for gradient updates
<code>num_leaves</code>	16 - 256	Model complexity; more leaves = more complex
<code>min_child_samples</code>	5 - 100	Min samples per leaf; prevents overfitting
<code>feature_fraction</code>	0.4 - 1.0	Fraction of features per tree (like RF)
<code>bagging_fraction</code>	0.4 - 1.0	Fraction of rows per tree (stochastic GBM)
<code>reg_alpha</code>	1e-4 - 10 (log)	L1 regularisation

reg_lambda	1e-4 - 10 (log)	L2 regularisation
------------	-----------------	-------------------

Each trial evaluates a 3-fold CV AUC. After OPTUNA_TRIALS (default 50) trials, the best parameters are passed to the full training run.

8. Full LightGBM Cross-Validation (train_lgbm_cv)

Out-of-fold (OOF) predictions

The 5-fold CV loop generates OOF predictions - each sample is predicted exactly once by a model that has never seen it during training. The final AUC is computed on these OOF predictions, giving an honest estimate of generalisation performance.

```
oof_preds = zeros(n_samples)
for fold in 5_folds:
    train model on 4 folds
    predict on held-out fold --> oof_preds[val_idx]
final_auc = roc_auc_score(y_true, oof_preds)
```

Early stopping

`lgb.early_stopping(50)` stops training if validation AUC does not improve for 50 consecutive rounds. This prevents overfitting without needing to fix the number of trees in advance. The validation set for early stopping is the held-out fold.

9. Evaluation & Model Interpretation

ROC Curve & AUC

Plots the True Positive Rate vs False Positive Rate at all classification thresholds. AUC (area under this curve) measures how well the model ranks defaulters above non-defaulters. AUC=0.5 is random; AUC=1.0 is perfect.

Precision-Recall Curve

More informative than ROC for imbalanced classes. Shows the trade-off between precision (of flagged applicants, what fraction actually default?) and recall (of all defaulters, what fraction are caught?).

Confusion Matrix

At the default threshold of 0.5, shows True Positives, False Positives, True Negatives, and False Negatives. The threshold can be adjusted - a lower threshold catches more defaulters but also flags more good applicants.

Feature Importance (Gain)

LightGBM records how much each feature reduces the loss function across all splits where it is used (gain). The top 30 features by gain are plotted as a horizontal bar chart.

SHAP Summary Plot

SHAP (SHapley Additive exPlanations) assigns a contribution score to each feature for each individual prediction. The summary plot shows:

- Each point is a sample; position on x-axis = SHAP value (impact on prediction).

- Colour = feature value (red = high, blue = low).
- Features are sorted by mean absolute SHAP value (most impactful at top).

This is far more informative than plain feature importance because it shows the direction and magnitude of each feature's effect.

10. Output Files

File	Description
eda_target_missing.png	Target distribution bar + top-30 missing rate chart
eda_correlation_heatmap.png	Pearson correlation heatmap for top 20 numeric features
model_comparison.png	Horizontal bar chart of CV AUC by model
model_comparison.csv	Table of all model CV AUC scores
roc_pr_lgbm.png	ROC curve and Precision-Recall curve (LightGBM OOF)
confusion_matrix.png	Confusion matrix at threshold=0.5
feature_importance.png	Top 30 features by LightGBM gain
shap_summary.png	SHAP beeswarm plot (top 20 features)
submission.csv	Kaggle submission file (requires application_test.csv)

11. How to Run

Install dependencies

```
pip install pandas numpy matplotlib seaborn scikit-learn
pip install lightgbm xgboost catboost optuna shap
```

Prepare data

```
# Download from Kaggle:
# https://www.kaggle.com/c/home-credit-default-risk/data
# Then:
mkdir data
mv application_train.csv data/
```

Run the pipeline

```
python home_credit.py
```

Key questions to investigate

- Which features are most predictive of default?
- Does adding bureau.csv and previous_application.csv improve AUC?
- How does LightGBM compare to logistic regression in AUC and interpretability?
- Are any features leaking future information?
- How does class imbalance affect the optimal classification threshold?
- Which types of customers are most often misclassified?
- Is the model stable across the 5 validation folds?